INT 315: Cluster Computing

Project Report

(SEMESTER July – December 2026)



LOVELY PROFESSIONAL UNIVERSITY

Coffee Shop Sales Analysis and Machine Learning Using PySpark

Submitted by

Amandeep Singh

Registration Number – 12315185

BTech CSE K4P23WR

Course Code: INT – 315

Under the Guidance of

(Pardeep Kumar (UID: 25237))

Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara

1. Project Title

Coffee Shop Sales Analysis and Machine Learning Using PySpark

2. Problem Statement

The sales of coffee shops produce significant amounts of transaction data on a daily basis. This data includes valuable information related to the needs of consumers, pricing, sales performance, purchasing behavior, and other details. Nevertheless, manual analysis of these logs may take quite much time and become challenging to uncover certain trends and relations.

The problem that is to be solved in this project is related to the analysis of the sales of coffee shops using distributed processing of data and machine learning approaches. The project employs the use of Apache Spark with PySpark for the purpose of analyzing transaction logs, performing exploratory data analysis, and applying regression, classification, and clustering algorithms. Apache Spark is a unifying framework used for the purposes of processing big data and machine learning [1].

This project will be focused on the investigation of sales trends, predicting of product categories on the basis of transaction characteristics, and clustering of products according to sales quantity, average price, and frequency of transactions.

Reference citation: [1] Apache Spark, *Apache Spark Documentation*. Available at: https://spark.apache.org/

3. Objectives of the Project

The principal aim of the current project is the analysis of coffee shops sales dataset using PySpark and machine learning techniques to obtain insights and valuable patterns.

The aims are as follows:

a) Data Preprocessing: To read, examine, preprocess, and transform the data set of coffee shops sales using PySpark.
b) Exploratory Data Analysis: To conduct an exploratory analysis of sales depending on the product category, store, time, and day of the week through statistics and plots.
c) Regression Modeling: To build a linear regression model for predicting the store daily revenues and assess the model's performance with R^2, RMSE, MAE.
d) Classification: To implement the models of logistic regression and decision tree classification for predicting the product categories based on transactions features.
e) Product Clustering: To perform clustering for the products based on total quantity of purchased items, average price of the products, and transaction frequency.
f) Evaluation of Models' Performance: To evaluate the implemented models using appropriate metrics.

4. Dataset Description

The dataset used in this project is the Coffee Shop Sales dataset, which contains transaction-level records of coffee shop sales. It consists of 149,116 records and 11 columns describing transactions, products, prices, and store locations.

The dataset includes the following attributes:

a) Transaction ID: A unique identifier associated with a transaction.
b) Transaction Date: The date on which the transaction occurred.
c) Transaction Time: The time at which the transaction occurred.
d) Transaction Quantity: The quantity of products purchased in a transaction.
e) Store ID: The identifier of the store where the transaction took place.
f) Store Location: The location of the store.

g) Product ID: The identifier of the product sold.
h) Unit Price: The price per unit of the product.
i) Product Category: The category to which the product belongs.
j) Product Type: The specific type of product sold.
k) Product Detail: Additional information describing the product.

The dataset was imported into a PySpark dataframe for further analysis. The revenue feature was computed through multiplication of the transaction quantity and the unit price. Attributes like date and time were also employed during the process of analyzing the sales data.

These features were used for exploratory analysis, daily revenue prediction, product classification, and clustering of the products.

5. Dataset Source

The data used in this assignment, named Coffee Shop Sales, can be found on Kaggle. The data include sales transactions details such as transaction date and time, quantity, product price, product category, and store location.

Dataset Name: Coffee Shop Sales

Source: Kaggle

Dataset Link: https://www.kaggle.com/datasets/keremkarayaz/coffee-shop-sales

The data were downloaded in CSV format and used as input for data processing, exploration, regression, classification, and clustering using PySpark.

6. Tools and Technologies used

The following tools and technologies were utilized to develop the Coffee Shop Sales Analysis and Machine Learning project.

a) Python: Applied as the main programming language for data manipulation, analysis, visualization, and machine learning tasks.
b) Apache Spark: Applied as the distributed computing platform for processing and analyzing the dataset of coffee shop sales [1].
c) PySpark: Applied to use Apache Spark functionalities in Python and perform data transformations, aggregation, and analysis operations.
d) Spark MLlib: Applied to perform machine learning using Linear Regression, Logistic Regression, Decision Tree Classification, and K-Means Clustering algorithms [2].
e) Jupyter Notebook: Applied to write and run the code in different notebooks for data preprocessing, analysis, regression, classification, and clustering tasks.
f) Visual Studio Code (VS Code): Applied as the development environment for managing and running the project files.
g) Pandas: Applied where needed for manipulating tabular data and data preparation for visualization.
h) Matplotlib and Seaborn: Applied to build charts, heatmaps, and other visualizations to analyze sales patterns and model results.
i) Java Development Kit (JDK 17): Applied to provide Java runtime environment required by Apache Spark.
j) Git and GitHub: Applied for version control and storing the project files.

References for this section:

[1] Apache Software Foundation, “Apache Spark Documentation.” Available at: https://spark.apache.org/

[2] Apache Software Foundation, “MLlib: Main Guide.” Available at: https://spark.apache.org/docs/latest/ml-guide.html

7. Implementation / Methodology

The implementation of the project was done through PySpark for the purpose of data preprocessing, exploratory data analysis, and machine learning. The entire process was executed using five steps which were all implemented in different Jupyter Notebooks.

7.1 Data Preparation

This first notebook was used to import and analyze the coffee shop sales dataset. The structure of the dataset, the total number of records, and missing data were evaluated to determine the quality and structure of the dataset.

The revenue for each sale was determined using the formula quantity times unit price. Dates and times were transformed to provide useful information. Lastly, sales were aggregated using date and store to produce a daily sales dataset at the store level.

7.2 Exploratory Data Analysis

The second notebook, 02_Data_Analysis.ipynb, was utilized to analyze the sales trends and relationships from the dataset.

The analysis consisted of sales per product category and store locations, trends of sales based on the time of day, and trends of sales based on the days of the week. Graphs and a correlation map were used for visualization of the results.

7.3 Regression Analysis

Linear Regression was used in the third notebook, 03_Regression.ipynb, for predicting the daily revenue for the store. The prepared dataset at the store level per day consisted of 543 records.

The dataset was split chronologically using June 1st, 2023 as a cutoff date, resulting in 453 training and 90 testing records. Evaluation metrics used in this case were R-squared (R^2), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE).

This helped in evaluating the performance of a basic regression model in predicting daily revenue using the selected features.

7.4 Classification

The fourth notebook, 04_Classification.ipynb, concentrated on prediction of the category of products based on the transaction features.

In preprocessing, categorical features were encoded, and the chosen features were compiled in a feature vector. Logistic Regression and Decision Tree Classifier algorithms for classification were used.

Accuracy, F1-score, weighted precision, and weighted recall metrics were used to evaluate the model. Additionally, a confusion matrix and heatmap were created.

7.5 Clustering

Notebook 05_Clustering.ipynb utilized K-Means clustering algorithm to form clusters based on the sales characteristics of the products.

The product data was aggregated by using the product IDs, product category, and product type. Three features were created: total units sold, average cost, and total number of transactions. These features were formed and then normalized before being used for K-Means algorithm with three clusters.

The clusters formed were then analyzed to see the difference between higher-cost and lower-sales products, high-sales and lower-cost products, and products in-between sales activity.

7.6 Overall Workflow

The entire implementation process involved the following order:

Dataset Loading -> Data Preparation -> Exploratory Data Analysis -> Regression -> Classification -> Clustering -> Evaluation and Interpretation.

8. Complete Code

The complete source code of the Coffee Shop Sales Analysis and Machine Learning project consists of five Jupyter Notebooks. Every notebook provides the source code that corresponds to a particular stage of the project, including data preparation, machine learning, and clustering.

The source code is provided in the following sequence:

8.3 01_Data_Preparation.ipynb - Data loading, dataset structure inspection, data quality control, creation of the revenue feature, extraction of date-time features, aggregation of daily sales per store

```
// Setting up Spark
import os
os.environ["JAVA_HOME"] = r"C:\Program Files\Java\jdk-17"
os.environ["PATH"] = os.environ["JAVA_HOME"] + r"\bin;" + os.environ["PATH"]
print("JAVA_HOME:", os.environ["JAVA_HOME"])

from pyspark.sql import SparkSession
spark = (
    SparkSession.builder
    .appName("CoffeeShopProject")
    .master("local[*]")
    .getOrCreate()
)
print("Spark version:", spark.version)
print("Spark session:", spark.sparkContext.appName)

// Loading the data
file_path = r"D:\Projects\Coffee Shop Sales - Cluster Computing\Dataset\Coffee Shop
Sales.csv"
df = spark.read.csv(
    file_path,
    header = True,
    inferSchema = True
)
print("Total rows:", df.count())
print("Total columns:", len(df.columns))

df.printSchema()


// Deriving Transaction Revenue
from pyspark.sql.functions import col, sum, avg, count
df = df.withColumn(
    "revenue",
    col("transaction_qty") * col("unit_price")
)

df.select(
    "transaction_id",
    "product_detail",
    "transaction_qty",
    "unit_price",
    "revenue"
).show(10, truncate=False)
```

```
category_sales = (
    df.groupBy("product_category")
      .agg(
          sum("revenue").alias("total_revenue"),
          sum("transaction_qty").alias("total_quantity"),
          count("*").alias("transaction_rows")
      )
      .orderBy(col("total_revenue").desc())
)

category_sales.show(truncate=False)


store_sales = (
    df.groupBy("store_location")
      .agg(
          sum("revenue").alias("total_revenue"),
          sum("transaction_qty").alias("total_quantity"),
          count("*").alias("transaction_rows")
      )
      .orderBy(col("total_revenue").desc())
)

store_sales.show(truncate=False)
```

8.3 02_Data_Analysis.ipynb - Exploration of the data, revenue analysis by product categories and store locations, sales patterns on hourly basis and depending on days of the week, visualization of correlations.

```
// Setting up Spark
import os

os.environ["JAVA_HOME"] = r"C:\Program Files\Java\jdk-17"
os.environ["PATH"] = os.environ["JAVA_HOME"] + r"\bin;" + os.environ["PATH"]

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("CoffeeShopDataAnalysis")
    .master("local[2]")
    .config("spark.ui.enabled", "false")
    .getOrCreate()
)

print("Spark Version:", spark.version)

// Loading the data
file_path = r"D:\Projects\Coffee Shop Sales - Cluster Computing\Dataset\Coffee Shop
Sales.csv"
df = spark.read.csv(
    file_path,
    header = True,
    inferSchema = True
)
print("Total Rows:", df.count())
print("Total Columns:", len(df.columns))
df.show(5, truncate=False)
```

// Revenue by Product Category

```
from pyspark.sql.functions import col, sum

df = df.withColumn(
    "revenue",
    col("transaction_qty") * col("unit_price")
)

category_revenue = (
    df.groupBy("product_category")
    .agg(sum("revenue").alias("total_revenue"))
    .orderBy(col("total_revenue").desc())
)

category_revenue.show(truncate=False)
```

// Sales by Hour

```
from pyspark.sql.functions import hour, col, sum

df = df.withColumn(
    "hour",
    hour(col("transaction_time"))
)

hourly_sales = (
    df.groupBy("hour")
    .agg(sum("revenue").alias("total_revenue"))
    .orderBy("hour")
)

hourly_sales.show(24)
```

// Sales by Day of Week

```
from pyspark.sql.functions import (
    to_date, dayofweek, col, sum
)

df = df.withColumn(
    "transaction_date",
    to_date(col("transaction_date"), "dd-MM-yyyy")
)


df = df.withColumn(
    "day_of_week",
    dayofweek(col("transaction_date"))
)


daily_week_sales = (
    df.groupBy("day_of_week")
    .agg(sum("revenue").alias("total_revenue"))
    .orderBy("day_of_week")
)

daily_week_sales.show()
```

```
// Correlation Analysis
import seaborn as sns
import matplotlib.pyplot as plt

numeric_df = df.select(
    "transaction_qty",
    "unit_price",
    "revenue",
    "hour"
).toPandas()

correlation_matrix = numeric_df.corr()

plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap - Coffee Shop Sales")
plt.tight_layout()
plt.show()
```

8.3 03_Regression.ipynb - Training and test datasets preparation, Linear Regression model training, predictions of daily sales per store, evaluation of the model via R^2, RMSE, and MAE.

```
// Setting up Spark
import os

os.environ["JAVA_HOME"] = r"C:\Program Files\Java\jdk-17"
os.environ["PATH"] = os.environ["JAVA_HOME"] + r"\bin;" + os.environ["PATH"]

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("CoffeeShopRegression")
    .master("local[2]")
    .config("spark.ui.enabled", "false")
    .getOrCreate()
)

print("Spark Version:", spark.version)
```

```
// Loading the data
file_path = r"D:\Projects\Coffee Shop Sales - Cluster Computing\Dataset\Coffee Shop
Sales.csv"
df = spark.read.csv(
    file_path,
    header = True,
    inferSchema = True
)
print("Total Rows:", df.count())
print("Total Columns:", len(df.columns))

df.show(5, truncate=False)
```

// Preparing and transaction and daily sales data

```
from pyspark.sql.functions import col, to_date, hour

# Correctly parse the transaction date
df = df.withColumn(
    "transaction_date",
    to_date(col("transaction_date"), "dd-MM-yyyy")
)

# Calculate revenue for each transaction
df = df.withColumn(
    "revenue",
    col("transaction_qty") * col("unit_price")
)

# Extract the transaction hour
df = df.withColumn(
    "hour",
    hour(col("transaction_time"))
)

print("Total rows:", df.count())
df.select(
    "transaction_date",
    "store_id",
    "transaction_qty",
    "unit_price",
    "revenue",
    "hour"
).show(5, truncate=False)

from pyspark.sql.functions import sum, count
daily_sales = (
    df.groupBy(
        "transaction_date",
        "store_id",
        "store_location"
    )
    .agg(
        sum("revenue").alias("daily_revenue"),
        count("*").alias("transaction_count")
    )
    .orderBy("transaction_date", "store_id")
)
print("Total daily records:", daily_sales.count())
daily_sales.show(10, truncate=False)

from pyspark.sql.functions import dayofweek, month, dayofmonth
daily_sales = (
    daily_sales
    .withColumn("day_of_week", dayofweek("transaction_date"))
    .withColumn("month", month("transaction_date"))
    .withColumn("day_of_month", dayofmonth("transaction_date"))
)

daily_sales.select(
    "transaction_date",
    "store_id",
    "day_of_week",
    "month",
    "day_of_month",
    "daily_revenue"
).show(10, truncate=False)
```

// Split data into training and testing sets

```
from pyspark.sql.functions import col, lit
train_data = daily_sales.filter(
    col("transaction_date") < lit("2023-06-01")
)

test_data = daily_sales.filter(
    col("transaction_date") >= lit("2023-06-01")
)

print("Training rows:", train_data.count())
print("Testing rows:", test_data.count())
```

// Train the Linear Regression Model

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml import Pipeline

feature_cols = [
    "store_id",
    "day_of_week",
    "month",
    "day_of_month"
]

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features"
)

lr = LinearRegression(
    featuresCol="features",
    labelCol="daily_revenue",
    predictionCol="predicted_revenue"
)

pipeline = Pipeline(stages=[assembler, lr])

lr_model = pipeline.fit(train_data)
```

// Predictions

```
predictions = lr_model.transform(test_data)

predictions.select(
    "transaction_date",
    "store_id",
    "daily_revenue",
    "predicted_revenue"
).show(15, truncate=False)
```

```
// Evaluating Model Performance
from pyspark.ml.evaluation import RegressionEvaluator

metrics = ["r2", "rmse", "mae"]

for metric in metrics:
    evaluator = RegressionEvaluator(
        labelCol="daily_revenue",
        predictionCol="predicted_revenue",
        metricName=metric
    )

    score = evaluator.evaluate(predictions)
    print(f"{metric.upper()}: {score:.4f}")

print("Linear Regression Model Evaluation")
print("----------------------------------")

for metric in ["r2", "rmse", "mae"]:
    evaluator = RegressionEvaluator(
        labelCol="daily_revenue",
        predictionCol="predicted_revenue",
        metricName=metric
    )
    print(f"{metric.upper()}: {evaluator.evaluate(predictions):.4f}"
```

8.4 04_Classification.ipynb - Features encoding and assembling, training of Logistic Regression and Decision Tree classifiers, classification metrics evaluation, creation of the confusion matrix.

```
// Setting up Spark
import os

os.environ["JAVA_HOME"] = r"C:\Program Files\Java\jdk-17"
os.environ["PATH"] = os.environ["JAVA_HOME"] + r"\bin;" + os.environ["PATH"]

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("CoffeeShopClassification")
    .master("local[2]")
    .config("spark.ui.enabled", "false")
    .getOrCreate()
)

print("Spark Session Initialized Successfully!")

// Loading the data
file_path = r"D:\Projects\Coffee Shop Sales - Cluster Computing\Dataset\Coffee Shop
Sales.csv"
df = spark.read.csv(
    file_path,
    header = True,
    inferSchema = True
)
print("Total Rows:", df.count())
print("Total Columns:", len(df.columns))

df.show(5, truncate=False)
```

// Prepare features and target labels

```
from pyspark.sql.functions import col, to_date, hour, dayofweek, month

df = df.withColumn(
    "transaction_date",
    to_date(col("transaction_date"), "dd-MM-yyyy")
)

# Create useful features
df = df.withColumn(
    "hour",
    hour(col("transaction_time"))
)

df = df.withColumn(
    "day_of_week",
    dayofweek(col("transaction_date"))
)

df = df.withColumn(
    "month",
    month(col("transaction_date"))
)

print("Dataset loaded successfully!")
print("Rows:", df.count())
print("Columns:", len(df.columns))

df.show(5, truncate=False)
```

// Preprocessing

```
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.sql.functions import col

# Remove rows where essential values are missing
df = df.dropna(subset=[
    "product_category",
    "transaction_qty",
    "unit_price",
    "hour",
    "day_of_week",
    "month",
    "store_location"
])

# Convert product category into numerical labels
label_indexer = StringIndexer(
    inputCol="product_category",
    outputCol="label",
    handleInvalid="keep"
)

# Convert store location into a numerical category
location_indexer = StringIndexer(
    inputCol="store_location",
    outputCol="store_location_index",
    handleInvalid="keep"
)
```

```
# Convert store location into one-hot encoded features
encoder = OneHotEncoder(
    inputCols=["store_location_index"],
    outputCols=["store_location_vector"]
)

print("Feature preparation stages created successfully!")
print("Rows available:", df.count())
print("Target column: product_category")


from pyspark.ml.feature import VectorAssembler

feature_cols = [
    "transaction_qty",
    "unit_price",
    "hour",
    "day_of_week",
    "month",
    "store_location_vector"
]

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features"
)

print("Feature assembler created successfully!")
print("Input features:", feature_cols)


// Pipeline
from pyspark.ml import Pipeline

# Create the preprocessing pipeline
preprocessing_pipeline = Pipeline(
    stages=[
        label_indexer,
        location_indexer,
        encoder,
        assembler
    ]
)

# Split the original data
train_data, test_data = df.randomSplit(
    [0.8, 0.2],
    seed=42
)

print("Training rows:", train_data.count())
print("Testing rows:", test_data.count())

# Fit preprocessing transformations on training data
preprocessing_model = preprocessing_pipeline.fit(train_data)

# Apply transformations to both datasets
train_processed = preprocessing_model.transform(train_data)
test_processed = preprocessing_model.transform(test_data)

print("Training data transformed successfully!")
print("Testing data transformed successfully!")
```

// Training and Evaluating Logistic Regression

```
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=50
)

lr_model = lr.fit(train_processed)

print("Logistic Regression model trained successfully!")

predictions = lr_model.transform(test_processed)

predictions.select(
    "product_category",
    "label",
    "prediction",
    "probability"
).show(10, truncate=False)

from pyspark.ml.evaluation import MulticlassClassificationEvaluator

metrics = ["accuracy", "f1", "weightedPrecision", "weightedRecall"]

for metric in metrics:
    evaluator = MulticlassClassificationEvaluator(
        labelCol="label",
        predictionCol="prediction",
        metricName=metric
    )

    score = evaluator.evaluate(predictions)
    print(f"{metric}: {score:.4f}")
```

// Training and Evaluating Decision Tree

```
from pyspark.ml.classification import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    featuresCol="features",
    labelCol="label",
    maxDepth=10,
    seed=42
)

dt_model = dt.fit(train_processed)

print("Decision Tree model trained successfully!")

dt_predictions = dt_model.transform(test_processed)

dt_predictions.select(
    "product_category",
    "label",
    "prediction"
).show(10, truncate=False)
```

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

metrics = ["accuracy", "f1", "weightedPrecision", "weightedRecall"]

for metric in metrics:
    evaluator = MulticlassClassificationEvaluator(
        labelCol="label",
        predictionCol="prediction",
        metricName=metric
    )

    score = evaluator.evaluate(dt_predictions)
    print(f"{metric}: {score:.4f}")
```

8.5 05_Clustering.ipynb - Aggregation of features at product level, standardization of features, K-Means clustering with three clusters.

```
// Setting up Spark
import os

os.environ["JAVA_HOME"] = r"C:\Program Files\Java\jdk-17"
os.environ["PATH"] = os.environ["JAVA_HOME"] + r"\bin;" + os.environ["PATH"]

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("CoffeeShopClustering")
    .master("local[2]")
    .config("spark.ui.enabled", "false")
    .getOrCreate()
)

print("Spark Session Initialized Successfully!")
```

```
// Loading the Data
file_path = r"D:\Projects\Coffee Shop Sales - Cluster Computing\Dataset\Coffee Shop
Sales.csv"
df = spark.read.csv(
    file_path,
    header = True,
    inferSchema = True
)
print("Total Rows:", df.count())
print("Total Columns:", len(df.columns))

df.show(5, truncate=False)
```

```
// Scaling Clustering Features
from pyspark.ml.feature import VectorAssembler, StandardScaler

# Select numerical features
feature_cols = [
    "total_quantity_sold",
    "average_price",
    "total_transactions"
]
```

```
# Combine features into a vector
assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="raw_features"
)

product_features = assembler.transform(product_data)

# Standardize features
scaler = StandardScaler(
    inputCol="raw_features",
    outputCol="scaled_features",
    withMean=True,
    withStd=True
)

scaler_model = scaler.fit(product_features)
scaled_data = scaler_model.transform(product_features)

print("Features scaled successfully!")

scaled_data.select(
    "product_id",
    "raw_features",
    "scaled_features"
).show(5, truncate=False)
```

```
// Train the K-Means model
from pyspark.ml.clustering import KMeans

kmeans = KMeans(
    featuresCol="scaled_features",
    predictionCol="cluster",
    k=3,
    seed=42
)

kmeans_model = kmeans.fit(scaled_data)

print("K-Means model trained successfully!")
```

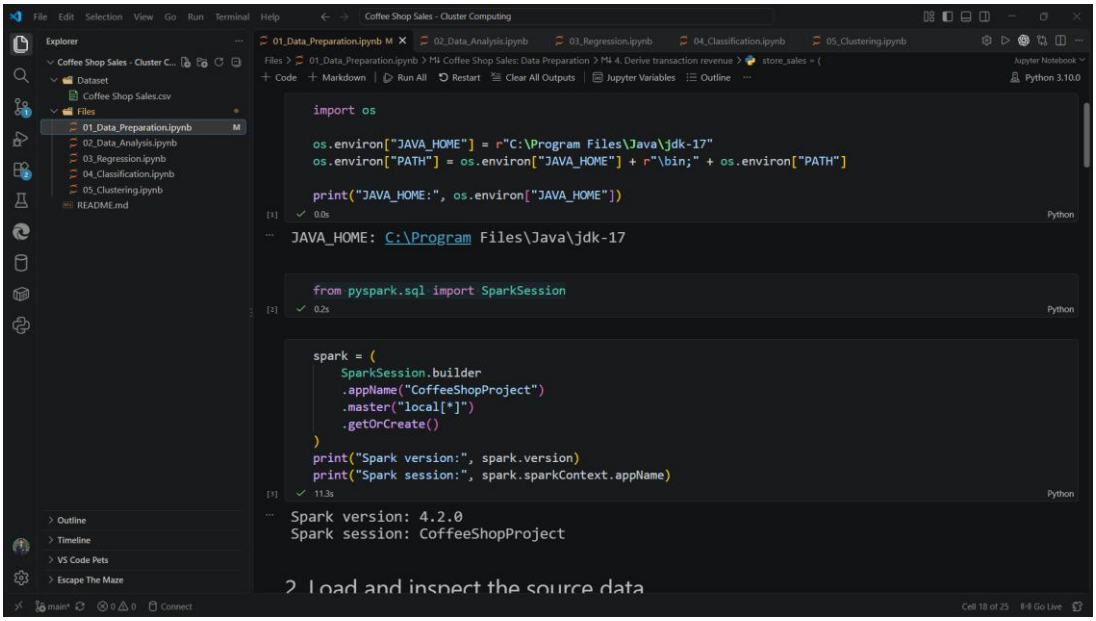
Thus, all notebooks represent the complete project implementation using Python, PySpark, and Spark MLlib.

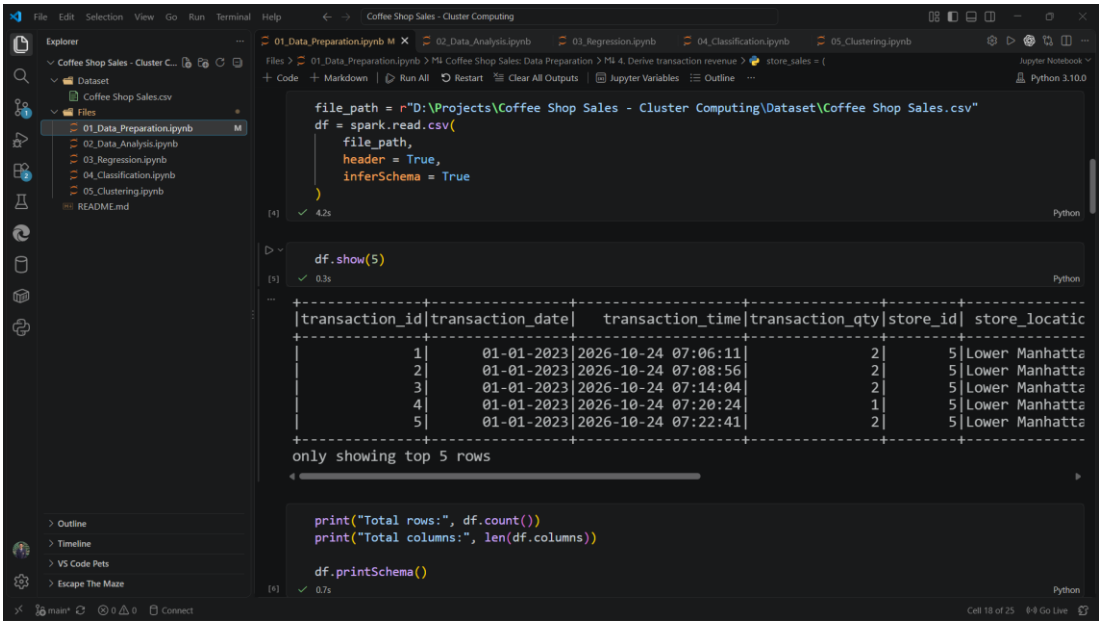
9. Execution Screenshots

In this section, screenshots are provided that depict the implementation of the Coffee Shop Sales Analysis and Machine Learning project. The screenshots illustrate that the code was indeed executed in the development environment and the various phases of the implementation process were performed.

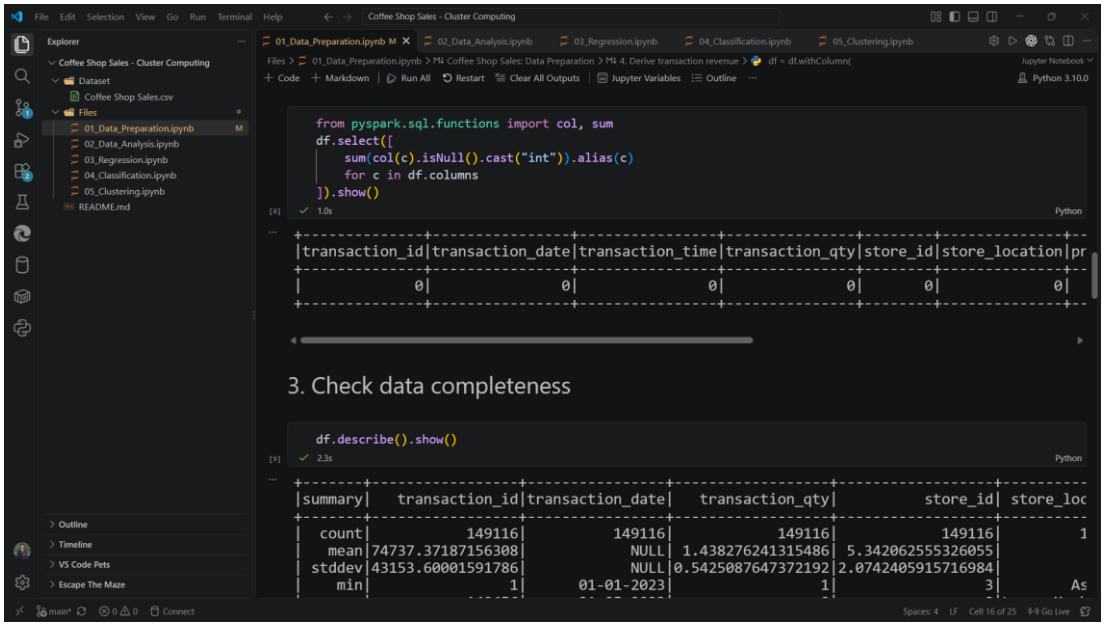
The screenshots related to the following phases include:

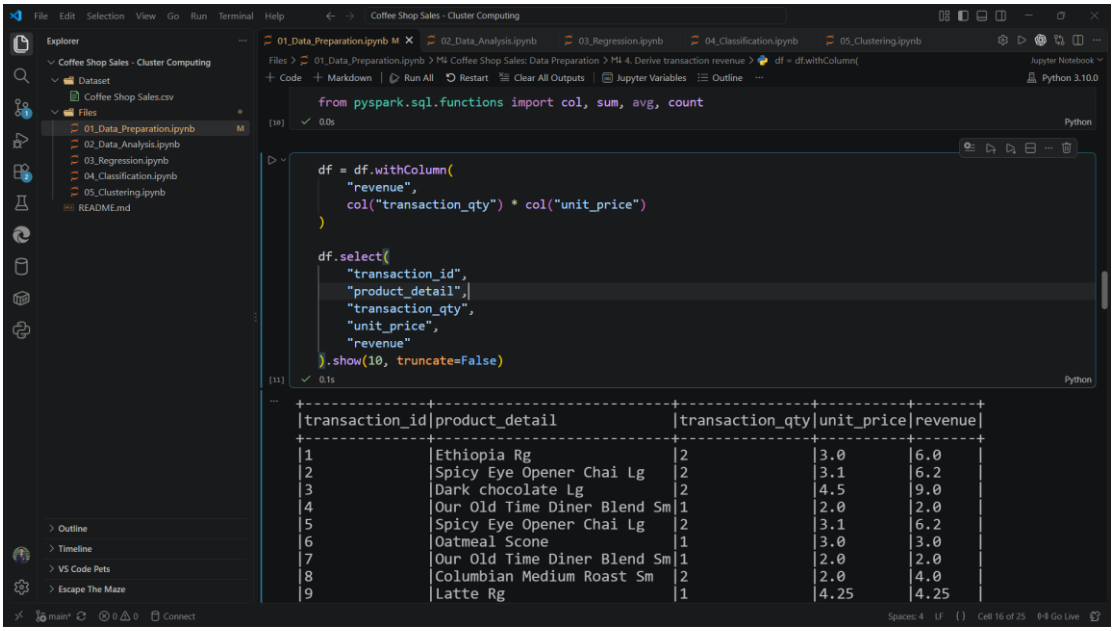
9.3Loading and inspecting the Coffee Shop Sales dataset.

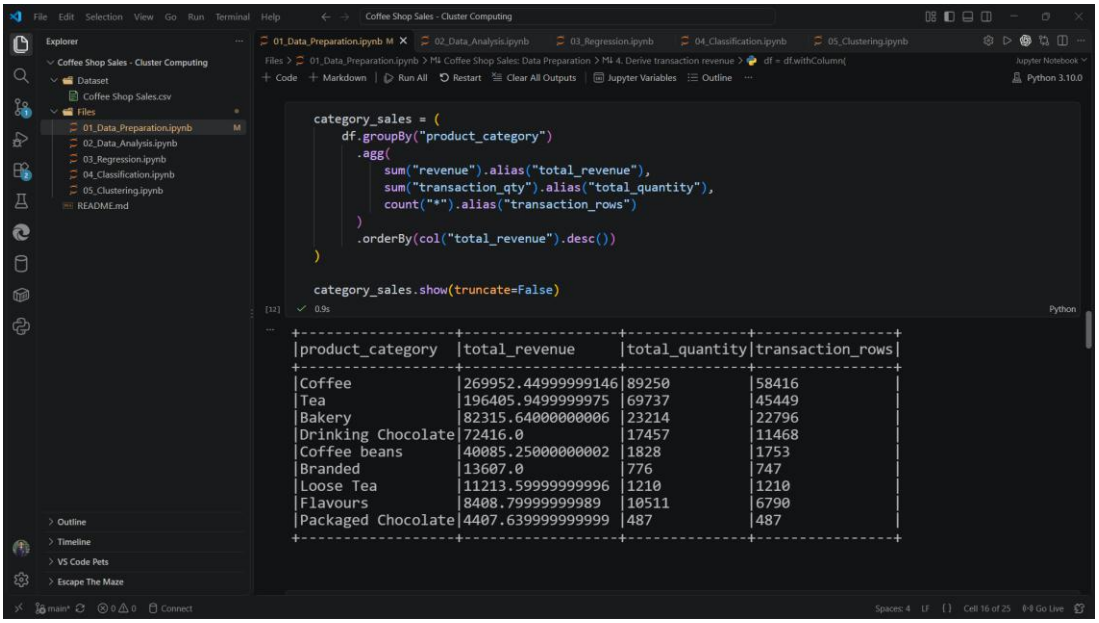




9.2 Data preparation and feature engineering.



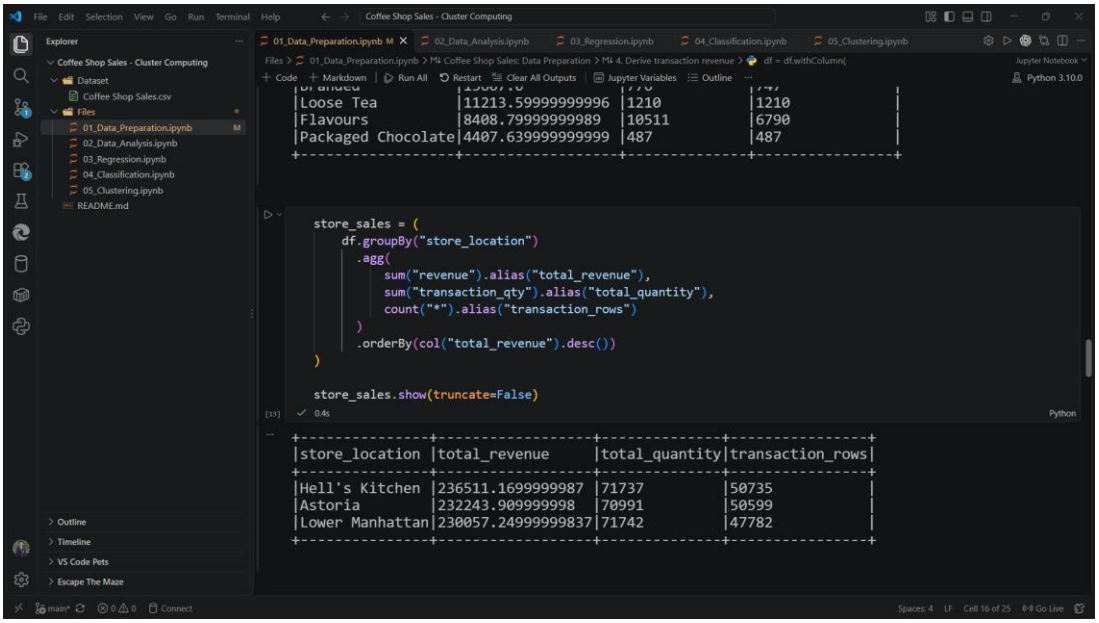




```
category_sales = (
    df.groupBy("product_category")
      .agg(
          sum("revenue").alias("total_revenue"),
          sum("transaction_qty").alias("total_quantity"),
          count("*").alias("transaction_rows")
      )
      .orderBy(col("total_revenue").desc())
)

category_sales.show(truncate=False)
```

```
+----------------+-----------------+--------------+----------------+
|product_category|total_revenue    |total_quantity|transaction_rows|
+----------------+-----------------+--------------+----------------+
|Coffee          |269952.44999999146|89250         |58416           |
|Tea             |196405.9499999975|69737         |45449           |
|Bakery          |82315.64000000006|23214         |22796           |
|Drinking Chocolate|72416.0        |17457         |11468           |
|Coffee beans    |40085.25000000002|1828          |1753            |
|Branded         |13607.0          |776           |747             |
|Loose Tea       |11213.59999999996|1210          |1210            |
|Flavours        |8408.79999999989 |10511         |6790            |
|Packaged Chocolate|4407.639999999999|487          |487             |
+----------------+-----------------+--------------+----------------+
```

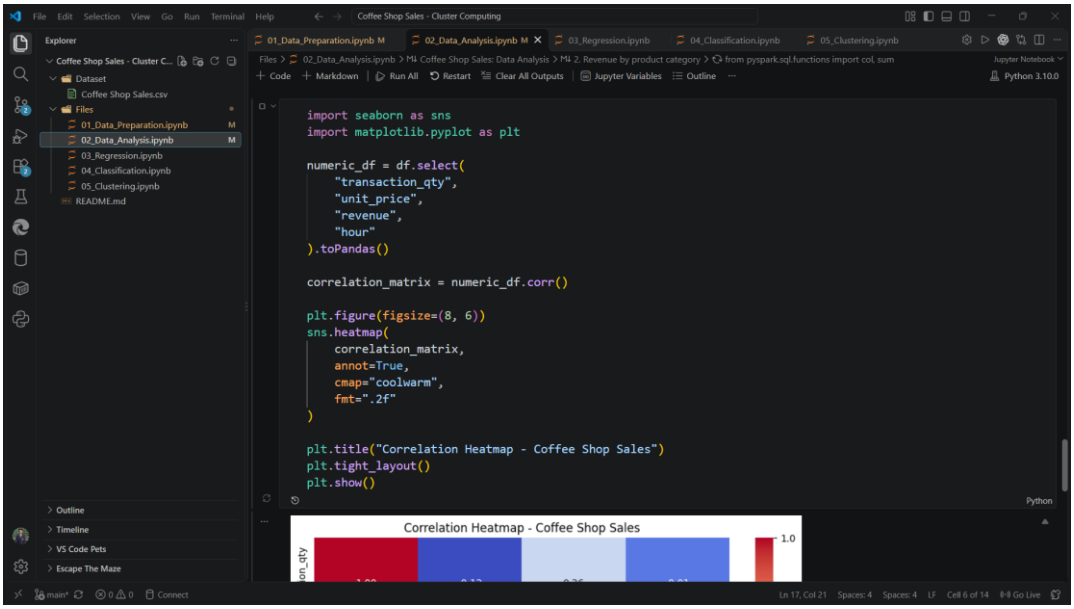


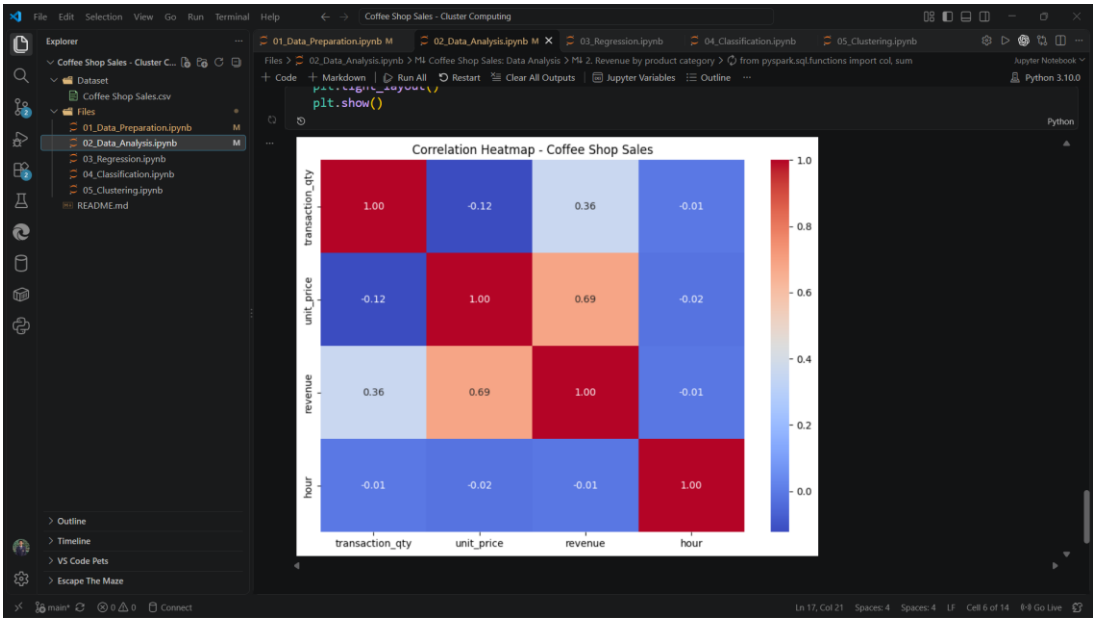
```
store_sales = (
    df.groupBy("store_location")
      .agg(
          sum("revenue").alias("total_revenue"),
          sum("transaction_qty").alias("total_quantity"),
          count("*").alias("transaction_rows")
      )
      .orderBy(col("total_revenue").desc())
)

store_sales.show(truncate=False)
```

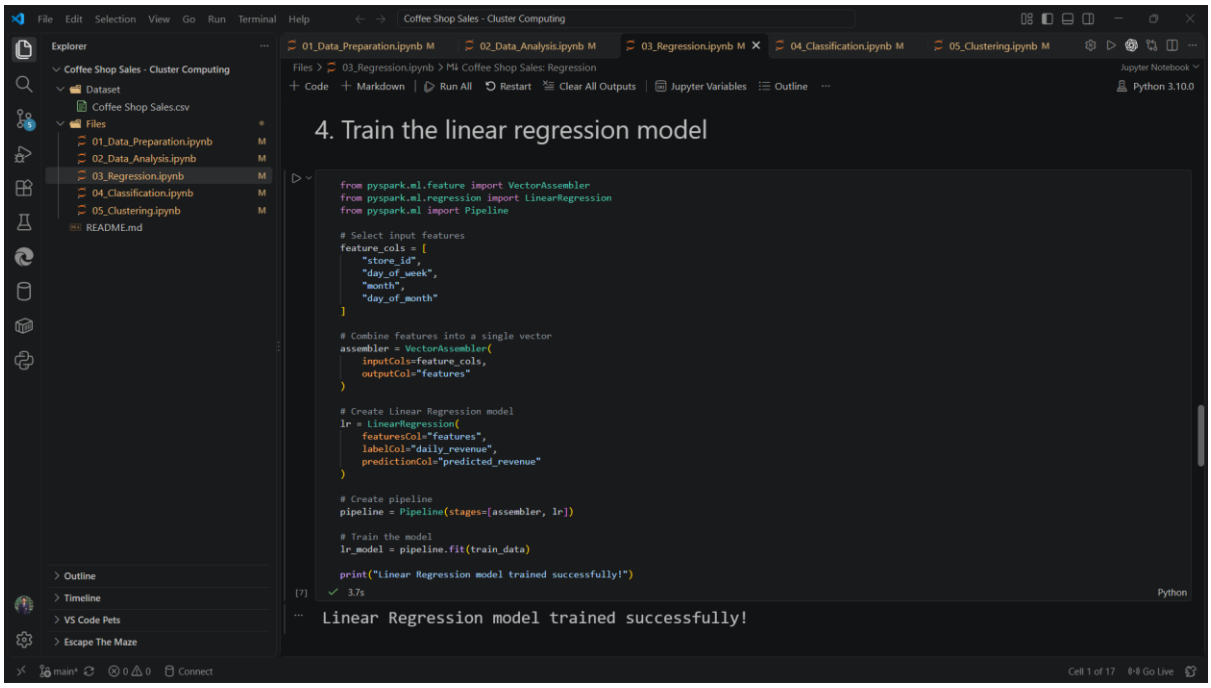
```
+--------------+-----------------+--------------+----------------+
|store_location|total_revenue    |total_quantity|transaction_rows|
+--------------+-----------------+--------------+----------------+
|Hell's Kitchen|236511.1699999987|71737         |50735           |
|Astoria       |232243.909999998 |70991         |50599           |
|Lower Manhattan|230057.24999999837|71742       |47782           |
+--------------+-----------------+--------------+----------------+
```

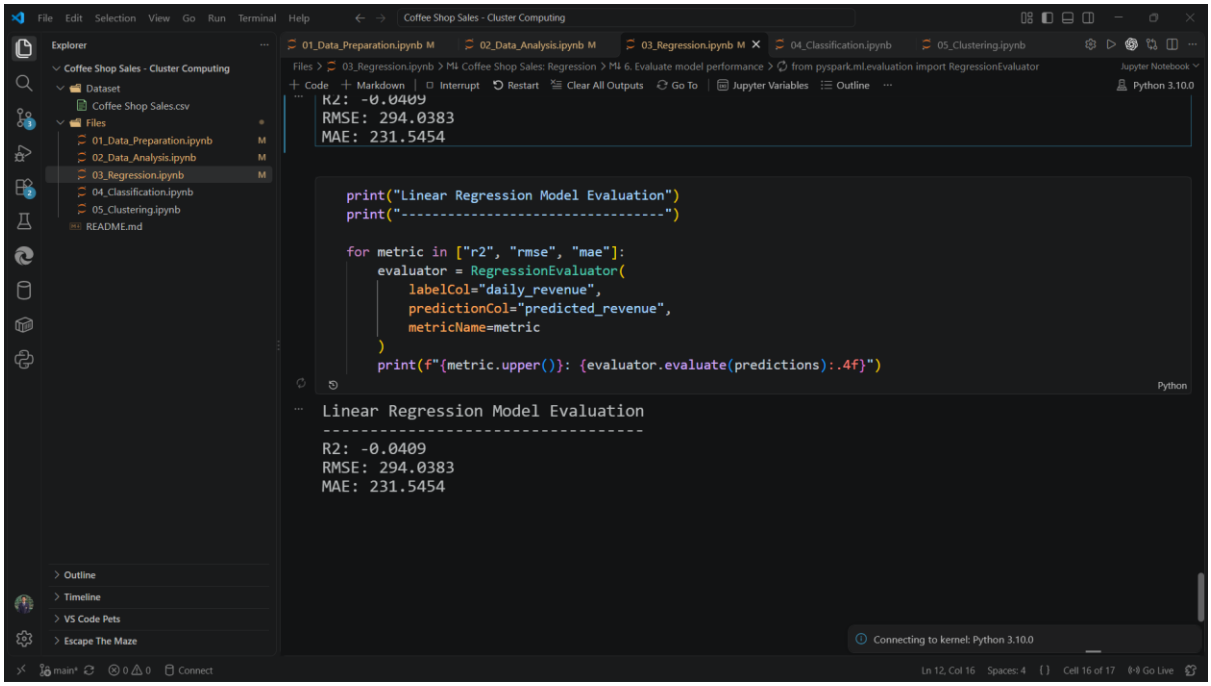
9.4 Exploratory data analysis.



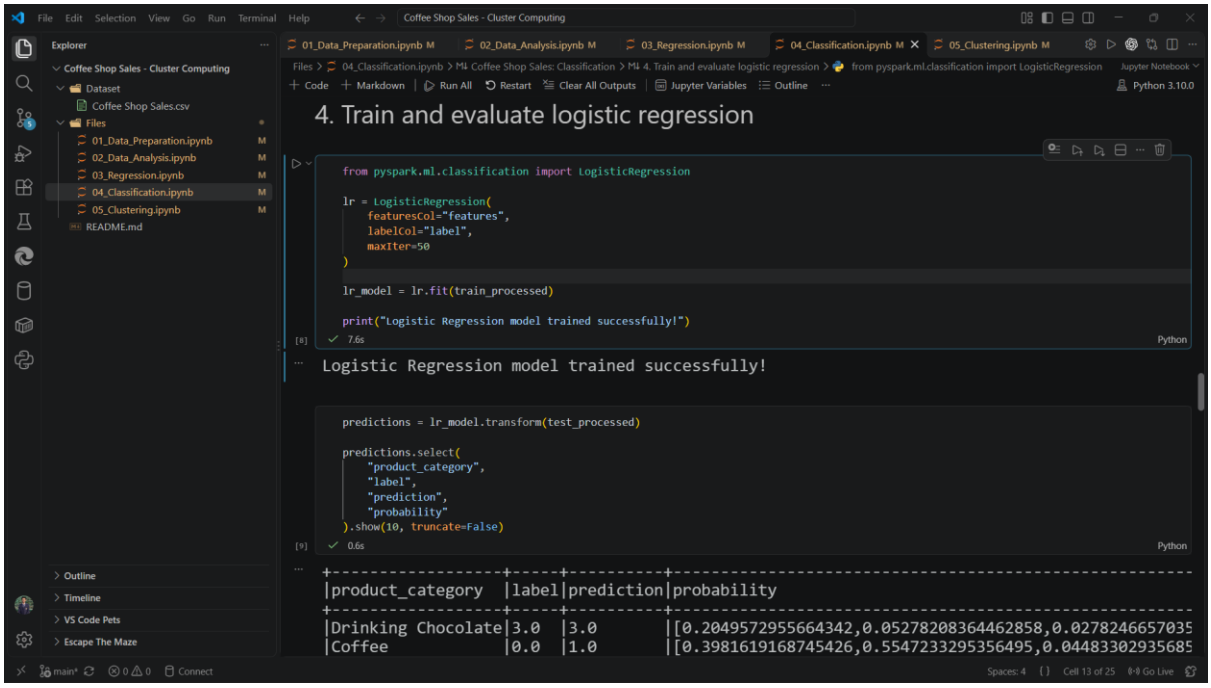


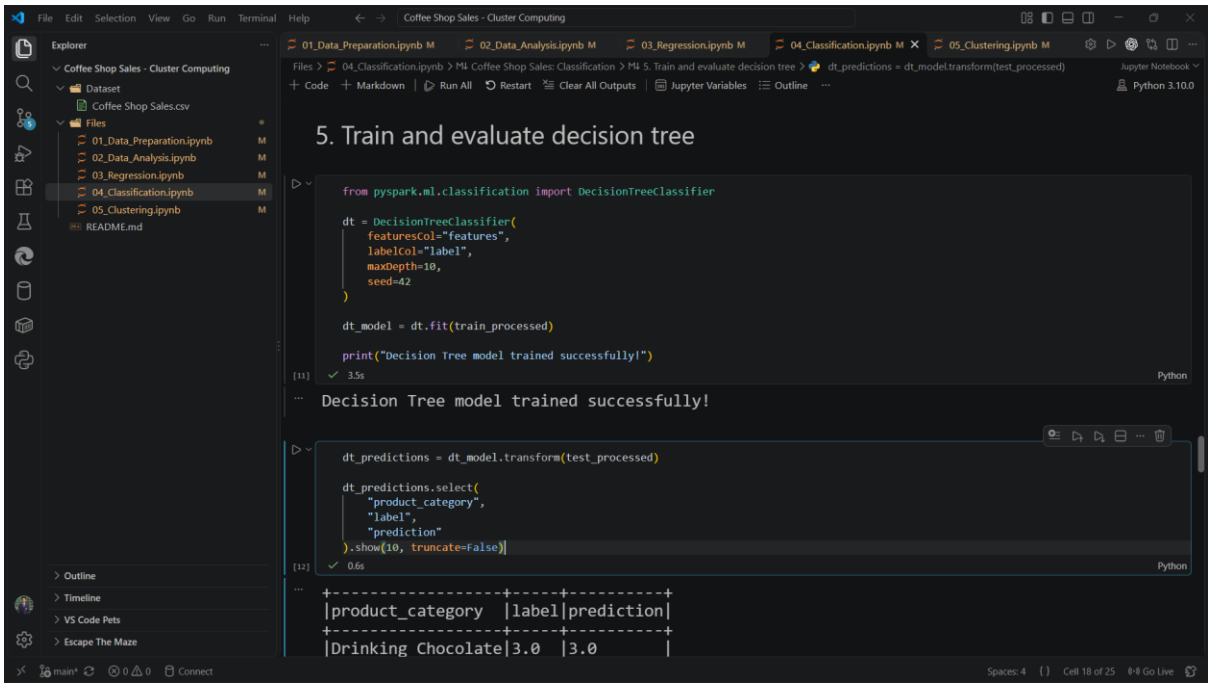
9.2 Training and evaluating Regression model



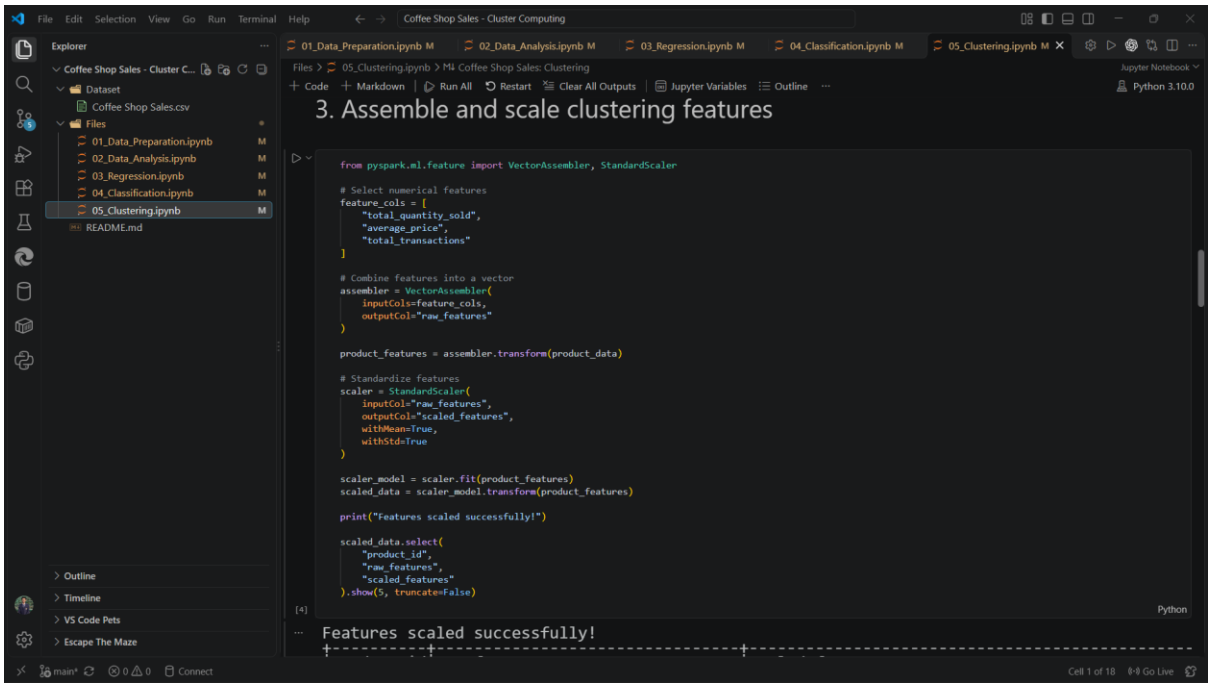


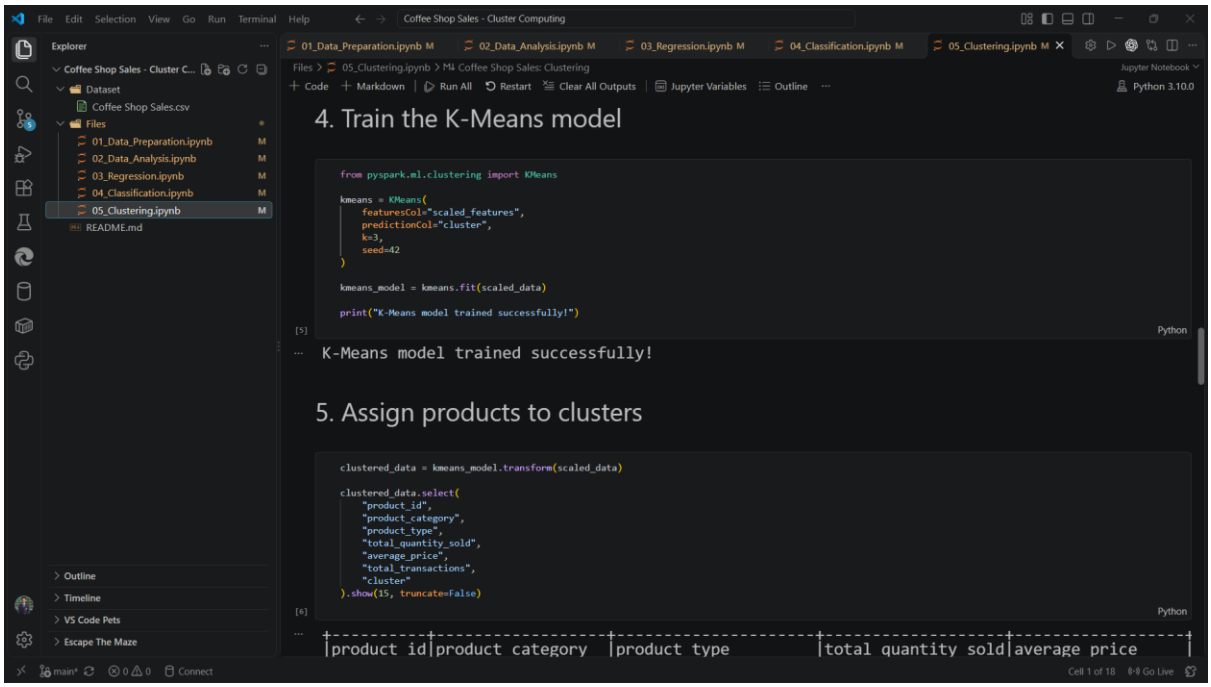
9.1 Training and evaluating the classification models.





9.6 Applying K-Means clustering to group products.



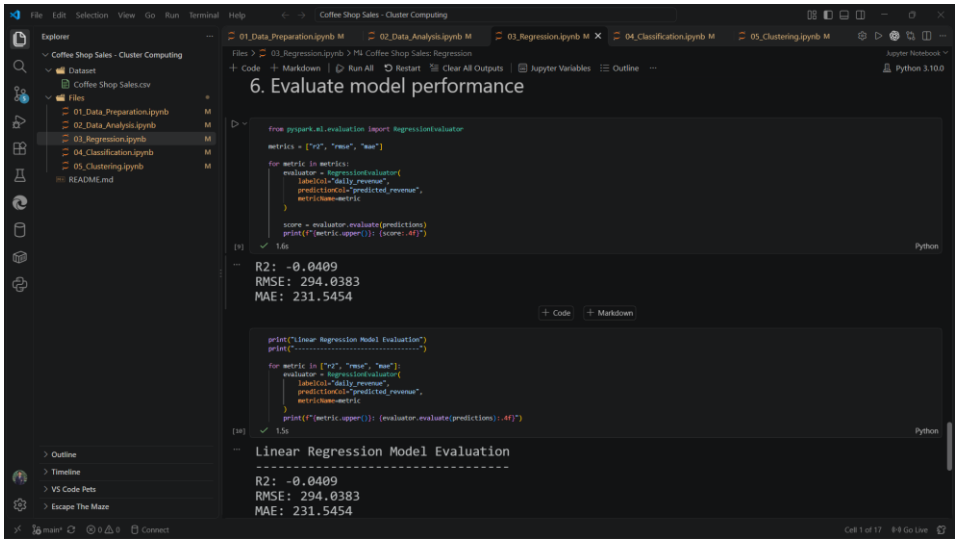


10. Output / Results Screenshots

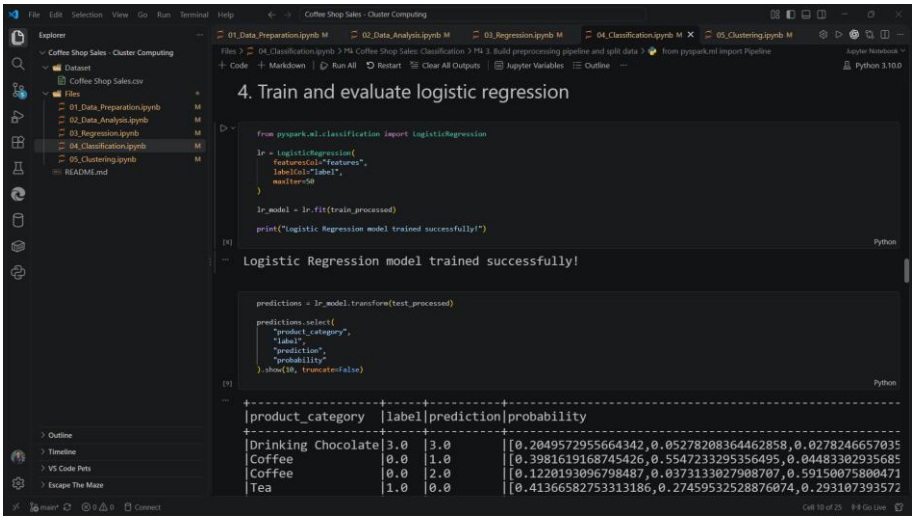
In this section, the outputs generated in the process of the analysis and machine learning in the Coffee Shop Sales project are presented. The pictures below show the outputs generated during the exploratory data analysis, regression, classification, and clustering using PySpark.

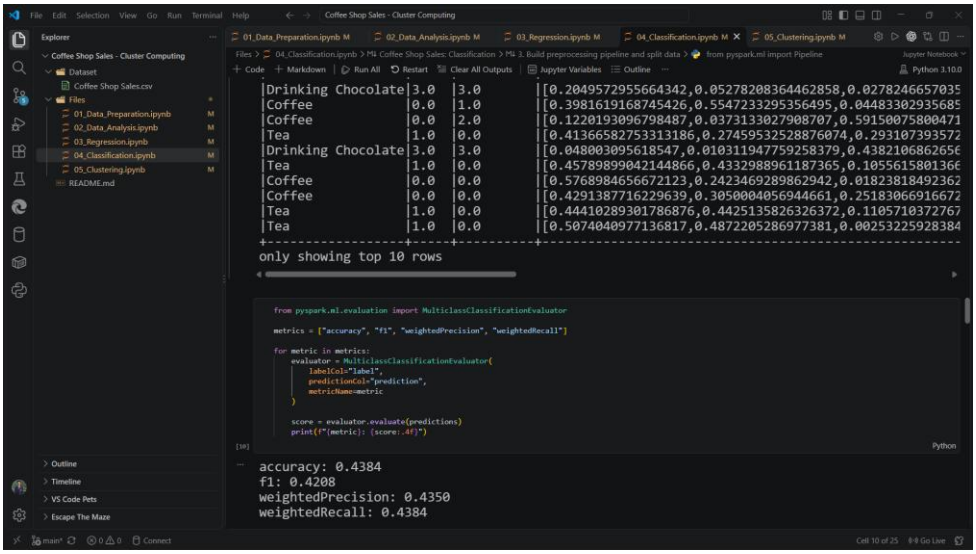
The outputs listed below have been provided:

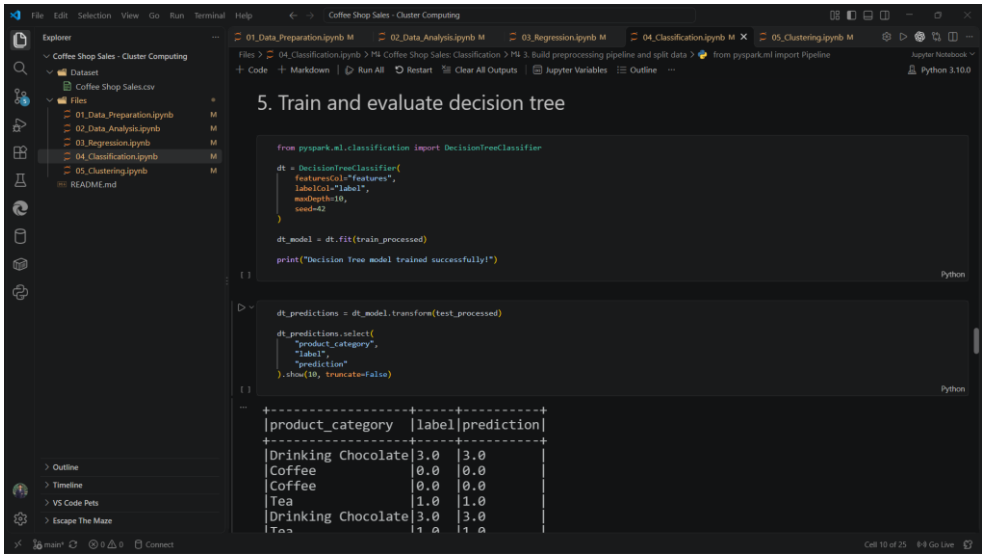
10.1 Regression Output

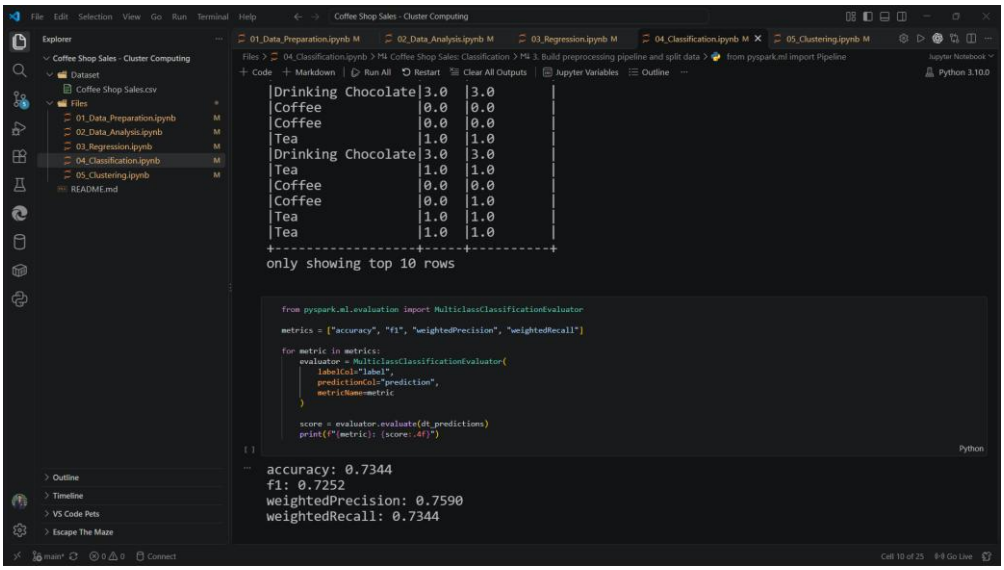


10.2 Classification Output

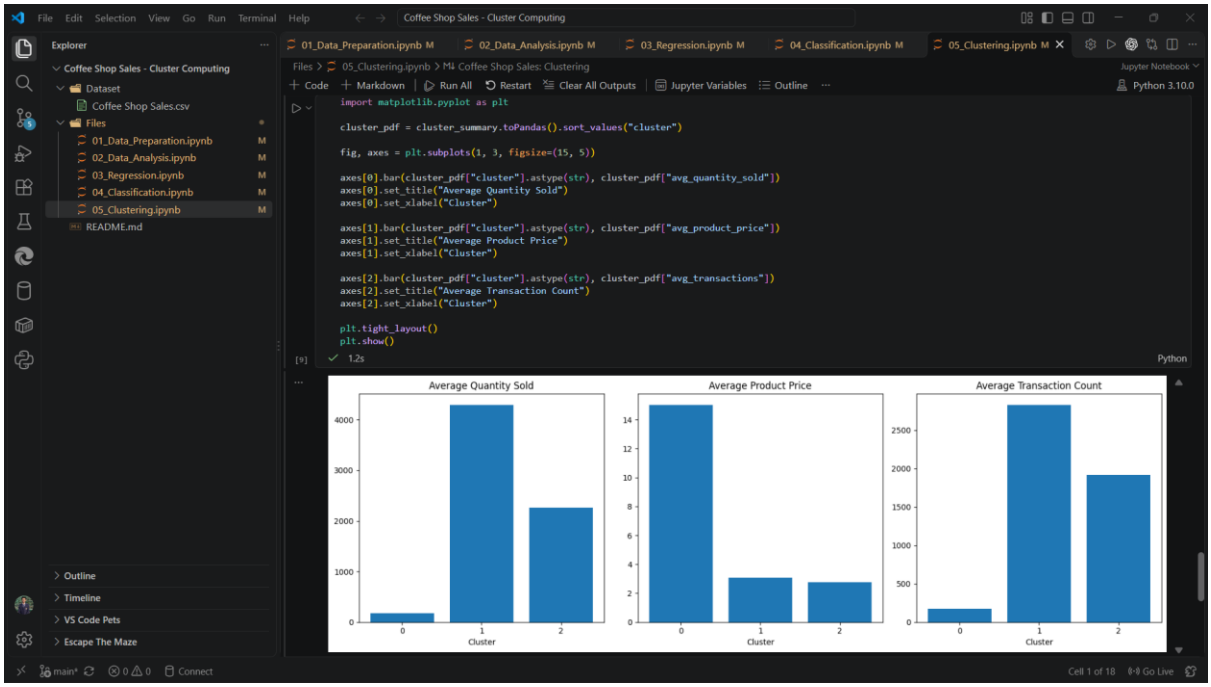



|Drinking Chocolate|3.0 |3.0 |[0.2049572955664342,0.05278208364462858,0.027824665703
|Coffee |0.0 |1.0 |[0.3981619168745426,0.5547233295356495,0.044833029356
|Coffee |0.0 |2.0 |[0.1220193096798487,0.037313027908707,0.59150075800471
|Tea |1.0 |0.0 |[0.41366582753313186,0.27459532528876074,0.293107393572
|Drinking Chocolate|3.0 |3.0 |[0.048003095618547,0.010311947759258379,0.438210686265
|Tea |1.0 |0.0 |[0.45789899042144866,0.4332988961187365,0.105561580136
|Coffee |0.0 |0.0 |[0.5768984656672123,0.2423469289862942,0.01823818492362
|Coffee |0.0 |0.0 |[0.4291387716229639,0.3050004056944661,0.25183066916672
|Tea |1.0 |0.0 |[0.4441028930178687 6,0.4425135826326372,0.1105710372767
|Tea |1.0 |0.0 |[0.5074040977136817,0.4872205286977381,0.0025322592838
only showing top 10 rows
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
metrics = ["accuracy", "f1", "weightedPrecision", "weightedRecall"]
for metric in metrics:
evaluator = MulticlassClassificationEvaluator(
labelCol="label",
predictionCol="prediction",
metricName=metric
)
score = evaluator.evaluate(predictions)
print(f"{metric}: {score:.4f}")
Python
accuracy: 0.4384
f1: 0.4208
weightedPrecision: 0.4350
weightedRecall: 0.4384


5. Train and evaluate decision tree
from pyspark.ml.classification import DecisionTreeClassifier
dt = DecisionTreeClassifier(
featuresCol="features",
labelCol="label",
maxDepth=10,
seed=42
)
dt_model = dt.fit(train_processed)
print("Decision Tree model trained successfully!")
Python
dt_predictions = dt_model.transform(test_processed)
dt_predictions.select(
"product_category",
"label",
"prediction"
).show(10, truncate=False)
Python
product_category	label	prediction
Drinking Chocolate	3.0	3.0
Coffee	0.0	0.0
Coffee	0.0	0.0
Tea	1.0	1.0
Drinking Chocolate	3.0	3.0


Drinking Chocolate	3.0	3.0
Coffee	0.0	0.0
Coffee	0.0	0.0
Tea	1.0	1.0
Drinking Chocolate	3.0	3.0
Tea	1.0	1.0
Coffee	0.0	0.0
Coffee	0.0	1.0
Tea	1.0	1.0
Tea	1.0	1.0
only showing top 10 rows
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
metrics = ["accuracy", "f1", "weightedPrecision", "weightedRecall"]
for metric in metrics:
evaluator = MulticlassClassificationEvaluator(
labelCol="label",
predictionCol="prediction",
metricName=metric
)
score = evaluator.evaluate(dt_predictions)
print(f"{metric}: {score:.4f}")
Python
accuracy: 0.7344
f1: 0.7252
weightedPrecision: 0.7590
weightedRecall: 0.7344

10.1 Clustering Results



11. Results Analysis

The results obtained from the Coffee Shop Sales project demonstrate how PySpark can be used for data preparation, exploratory analysis, and machine learning.

11.1 Regression Analysis

The Linear Regression model was evaluated using R-squared (R^2), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE). The model achieved an R^2 of -0.0409, an RMSE of 294.0383, and an MAE of 231.5454.

11.2 Classification Analysis

Two classification algorithms were compared to predict product categories.

Logistic Regression achieved an accuracy of 0.4384, an F1-score of 0.4208, weighted precision of 0.4350, and weighted recall of 0.4384.

The Decision Tree Classifier achieved an accuracy of 0.7344, an F1-score of 0.7252, weighted precision of 0.7590, and weighted recall of 0.7344.

11.3 Clustering Analysis

K-Means clustering divided products into three groups based on total quantity sold, average price, and total transaction count.

12. Conclusion

In this project, it has been shown how the technologies Apache Spark and PySpark could be used for analyzing the Coffee Shop Sales data and implementing machine learning methods. This project included the stages of data preparation, exploratory data analysis, regression, classification, and clustering. These stages were completed in five Jupyter Notebooks.

At the stage of data preparation and exploration, data about transactions were analysed to calculate such variables as revenue and time variables and find sales patterns for different product types, store locations, and time frames.

The results of machine learning experiments were not equal. For example, the Linear Regression model gave $R^2 = -0.0409$ and, hence, had poor prediction abilities regarding predicting daily store revenue based on the chosen features. In the case of the classification experiment, the Decision Tree Classifier achieved 73.44% accuracy, being more effective than Logistic Regression, which gave only 43.84% accuracy. In the clustering experiment, the K-Means method divided the products into three clusters based on the sales quantity, average price, and transactions number.

Thus, the current project helped me obtain practical skills related to the distributed data processing and machine learning with the help of PySpark.